

Digital Assignment

Plant Diseases Detection

Name: Soumyadeep Ganguly

Reg No: 24MDT0082

```
In [46]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import gc
import os
import random
import copy
import time
from PIL import Image
from pathlib import Path
from collections import Counter
from tqdm.notebook import tqdm
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torchsummary import summary
```

```
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models, utils, datasets
```

```
In [26]: DATA_DIR = "Plant Diseases Dataset/New Plant Diseases Dataset(Augmented)/train"
ROOT_DIR = "Plant Diseases Dataset/New Plant Diseases Dataset(Augmented)"
OUTPUT_DIR = "Plant_Diseases_Output"
IMG_SIZE = 256
BATCH_SIZE = 64
NUM_WORKERS = 4
SEED = 42
EPOCHS = 8
LR = 1e-4

train_dir = os.path.join(ROOT_DIR, 'train')
valid_dir = os.path.join(ROOT_DIR, 'valid')

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)
```

Device: cuda

Total Samples and Classes

```
In [9]: def discover_dataset(data_dir):
data_dir = Path(data_dir)
classes = sorted([d.name for d in data_dir.iterdir() if d.is_dir()])
class_to_idx = {c: i for i, c in enumerate(classes)}
files = []
labels = []
for c in classes:
    p = data_dir / c
    for ext in ('*.jpg', '*.jpeg', '*.png', '*.bmp'):
        for f in p.glob(ext):
            files.append(str(f))
            labels.append(class_to_idx[c])
```

```

    return files, labels, classes

files, labels, classes = discover_dataset(DATA_DIR)
print(f'Total images: {len(files)}')
print(f'Number of classes: {len(classes)}')

# class distribution
dist = Counter(labels)
class_counts = {classes[i]: dist[i] for i in sorted(dist.keys())}
class_counts = dict(sorted(class_counts.items(), key=lambda x: x[1], reverse=True))

pd.Series(class_counts).head(20)

```

Total images: 70295

Number of classes: 38

```

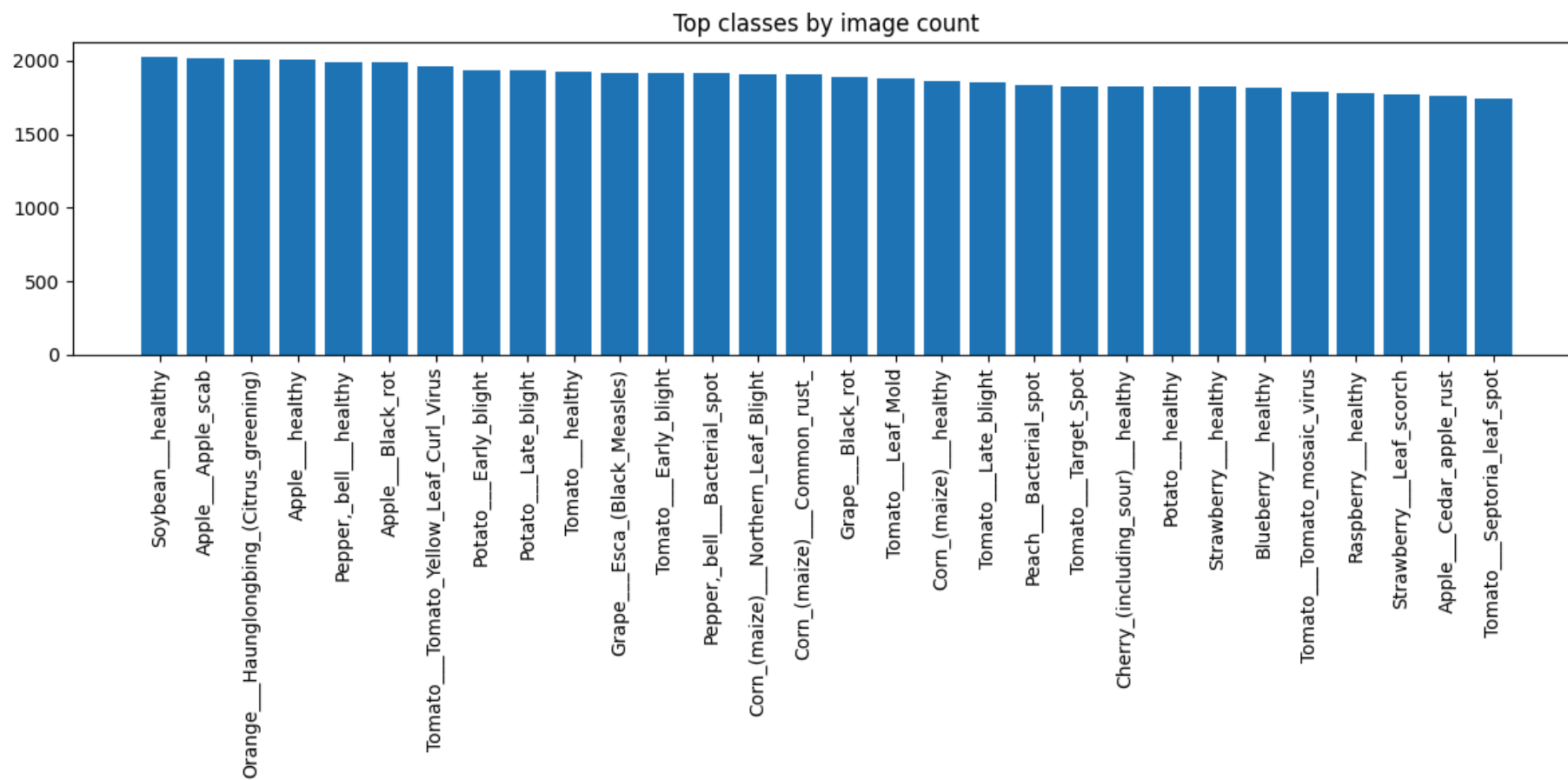
Out[9]: Soybean__healthy          2022
        Apple__Apple_scab        2016
        Orange__Haunglongbing_(Citrus_greening)  2010
        Apple__healthy           2008
        Pepper,_bell__healthy     1988
        Apple__Black_rot          1987
        Tomato__Tomato_Yellow_Leaf_Curl_Virus  1961
        Potato__Early_blight      1939
        Potato__Late_blight       1939
        Tomato__healthy           1926
        Grape__Esca_(Black_Measles)  1920
        Tomato__Early_blight      1920
        Pepper,_bell__Bacterial_spot  1913
        Corn_(maize)__Northern_Leaf_Blight  1908
        Corn_(maize)__Common_rust_  1907
        Grape__Black_rot          1888
        Tomato__Leaf_Mold         1882
        Corn_(maize)__healthy     1859
        Tomato__Late_blight       1851
        Peach__Bacterial_spot     1838
dtype: int64

```

Class Distribution and Sample Images

```
In [10]: plt.figure(figsize=(12,6))
vals = list(class_counts.values())
keys = list(class_counts.keys())
plt.bar(range(min(30, len(vals))), vals[:30])
plt.xticks(range(min(30, len(vals))), keys[:30], rotation=90)
plt.title('Top classes by image count')
plt.tight_layout()
plt.show()

# show random samples from some classes
NUM_COLS = 5
NUM_ROWS = 3
plt.figure(figsize=(NUM_COLS*3, NUM_ROWS*3))
for i, cls in enumerate(list(class_counts.keys())[:NUM_ROWS]):
    cls_dir = Path(DATA_DIR)/cls
    imgs = list(cls_dir.glob('*.jpg')) + list(cls_dir.glob('*.png'))
    for j in range(NUM_COLS):
        idx = i*NUM_COLS + j + 1
        plt.subplot(NUM_ROWS, NUM_COLS, idx)
        if imgs:
            img_p = random.choice(imgs)
            img = Image.open(img_p).convert('RGB')
            plt.imshow(img)
            plt.title(cls.replace('_', ' '))
            plt.axis('off')
plt.tight_layout()
plt.show()
```



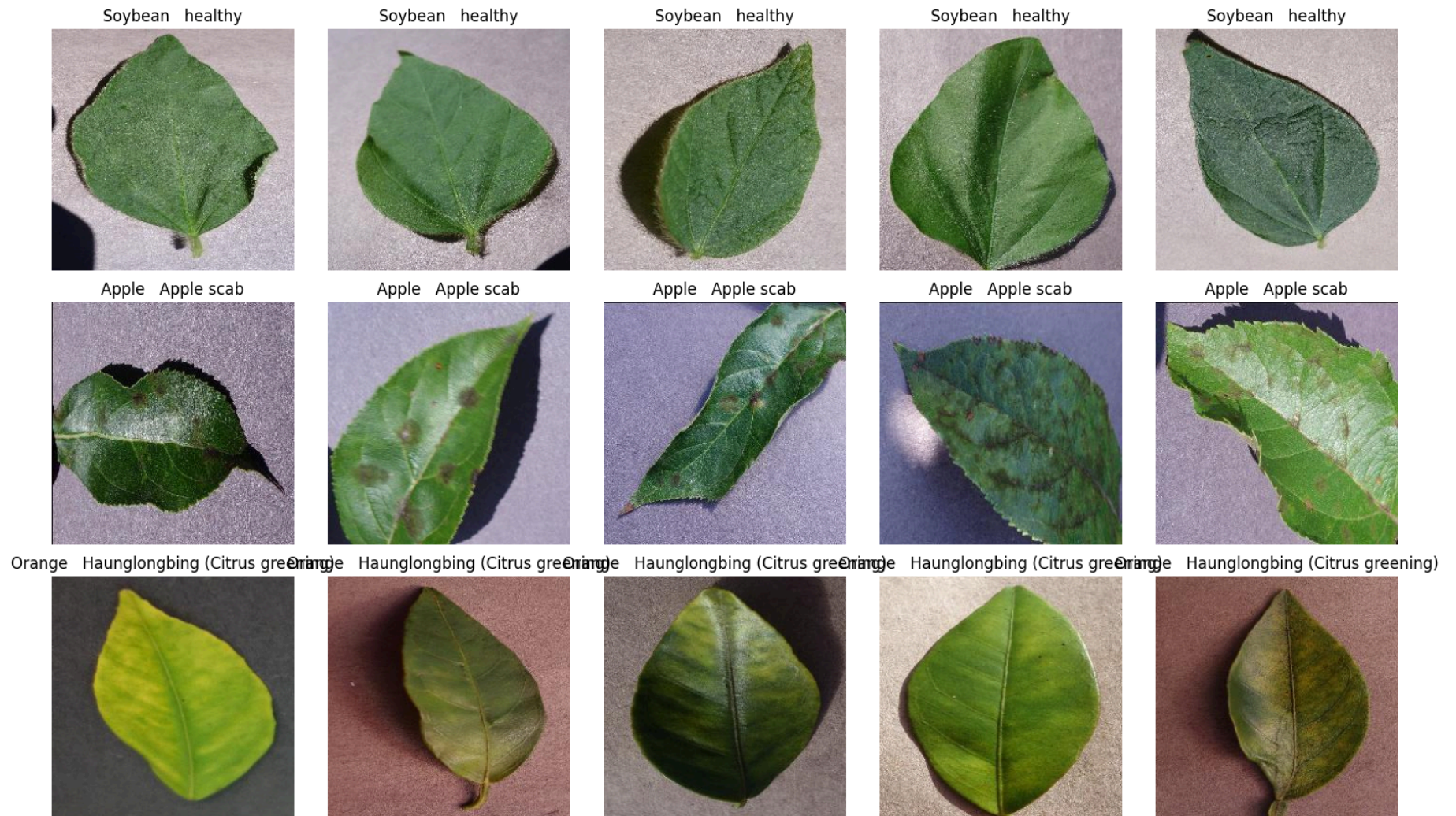


Image Size Distribution and Corrupted Checks

```
In [5]: widths = []
        heights = []
        corrupted = []
```

```
print("Checking image dimensions...")

for i, f in enumerate(files):
    try:
        with Image.open(f) as im:
            w, h = im.size
            widths.append(w)
            heights.append(h)
    except Exception:
        corrupted.append(f)

print("Done!")
print("Corrupted images:", len(corrupted))
```

Checking image dimensions...

Done!

Corrupted images: 0

Splitting Train and Test Set

```
In [11]: train_files, test_files, train_labels, test_labels = train_test_split(
        files, labels, test_size=0.2, stratify=labels, random_state=SEED)

val_files, test_files, val_labels, test_labels = train_test_split(
    test_files, test_labels, test_size=0.5, stratify=test_labels, random_state=SEED)

print('Split sizes ->', 'train:', len(train_files), 'val:', len(val_files), 'test:', len(test_files))
```

Split sizes -> train: 56236 val: 7029 test: 7030

Dataset Class

```
In [27]: train_transforms = transforms.Compose([
    transforms.Resize((128, 128)),          # Resize to fixed size
    transforms.RandomHorizontalFlip(p=0.5), # Flip horizontally
    transforms.RandomRotation(20),          # Rotate within 20 degrees
```

```

    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2), # Random color changes
    transforms.ToTensor(), # Convert to tensor
    transforms.Normalize((0.5,),(0.5,)) # Normalize
])

# Validation transforms (no augmentation here)
valid_transforms = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize((0.5,),(0.5,))
])

# Apply to datasets
train_data = datasets.ImageFolder(root=train_dir, transform=train_transforms)
valid_data = datasets.ImageFolder(root=valid_dir, transform=valid_transforms)

```

```

In [28]: # Create DataLoaders
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
valid_loader = DataLoader(valid_data, batch_size=BATCH_SIZE, shuffle=False)

print(f"Number of training images: {len(train_data)}")
print(f"Number of validation images: {len(valid_data)}")
print(f"Classes: {train_data.classes}")

```

Number of training images: 70295

Number of validation images: 17572

Classes: ['Apple__Apple_scab', 'Apple__Black_rot', 'Apple__Cedar_apple_rust', 'Apple__healthy', 'Blueberry__healthy', 'Cherry_(including_sour)__Powdery_mildew', 'Cherry_(including_sour)__healthy', 'Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot', 'Corn_(maize)__Common_rust_', 'Corn_(maize)__Northern_Leaf_Blight', 'Corn_(maize)__healthy', 'Grape__Black_rot', 'Grape__Esca_(Black_Measles)', 'Grape__Leaf_blight_(Isariopsis_Leaf_Spot)', 'Grape__healthy', 'Orange__Haunglongbing_(Citrus_greening)', 'Peach__Bacterial_spot', 'Peach__healthy', 'Pepper,bell__Bacterial_spot', 'Pepper,bell__healthy', 'Potato__Early_blight', 'Potato__Late_blight', 'Potato__healthy', 'Raspberry__healthy', 'Soybean__healthy', 'Squash__Powdery_mildew', 'Strawberry__Leaf_scorch', 'Strawberry__healthy', 'Tomato__Bacterial_spot', 'Tomato__Early_blight', 'Tomato__Late_blight', 'Tomato__Leaf_Mold', 'Tomato__Septoria_leaf_spot', 'Tomato__Spider_mites Two-spotted_spider_mite', 'Tomato__Target_Spot', 'Tomato__Tomato_Yellow_Leaf_Curl_Virus', 'Tomato__Tomato_mosaic_virus', 'Tomato__healthy']

Model Building: Simple CNN Model

```
In [29]: class PlantDiseaseCNN(nn.Module):
    def __init__(self, num_classes):
        super(PlantDiseaseCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 32 * 32, 256) # assuming input (128x128)
        self.fc2 = nn.Linear(256, num_classes)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(x.size(0), -1) # flatten
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)
        return x
```

```
In [ ]: # Initialize model
num_classes = len(train_data.classes)
model = PlantDiseaseCNN(num_classes).to(device)

# Loss & Optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
In [47]: summary(model, input_size=(3, 128, 128))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 32, 128, 128]	896
MaxPool2d-2	[-1, 32, 64, 64]	0
Conv2d-3	[-1, 64, 64, 64]	18,496
MaxPool2d-4	[-1, 64, 32, 32]	0
Linear-5	[-1, 256]	16,777,472
Dropout-6	[-1, 256]	0
Linear-7	[-1, 38]	9,766

=====

Total params: 16,806,630
Trainable params: 16,806,630
Non-trainable params: 0

Input size (MB): 0.19
Forward/backward pass size (MB): 7.50
Params size (MB): 64.11
Estimated Total Size (MB): 71.80

Training

```
In [31]: epochs = 50
patience = 7
best_val_acc = 0.0
best_model_wts = copy.deepcopy(model.state_dict())
early_stop_counter = 0

train_losses, val_losses = [], []
train_accuracies, val_accuracies = [], []

for epoch in range(epochs):
    print(f"Epoch {epoch+1} ===== Training Started -----")
    # ----- Training -----
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    batch_counter = 0
```

```
print(len(train_loader) )

for images, labels in train_loader:
    batch_counter += 1
    images, labels = images.to(device), labels.to(device)

    print(f"\rEpoch {epoch+1} --- Image {batch_counter} Loaded", end="", flush=True)

    optimizer.zero_grad()
    outputs = model(images)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()

    print(f"\r{batch_counter} pass done!", end="", flush=True)

    running_loss += loss.item()
    _, predicted = torch.max(outputs, 1)
    total += labels.size(0)
    correct += (predicted == labels).sum().item()

train_acc = 100 * correct / total
train_loss = running_loss / len(train_loader)

# ----- Validation -----
model.eval()
val_running_loss, val_correct, val_total = 0.0, 0, 0
with torch.no_grad():
    for images, labels in valid_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        val_running_loss += loss.item()
        _, predicted = torch.max(outputs, 1)
        val_total += labels.size(0)
        val_correct += (predicted == labels).sum().item()

val_acc = 100 * val_correct / val_total
val_loss = val_running_loss / len(valid_loader)

# history
```

```
train_losses.append(train_loss)
val_losses.append(val_loss)
train_accuracies.append(train_acc)
val_accuracies.append(val_acc)

# ----- Model Checkpoint -----
if val_acc > best_val_acc:
    best_val_acc = val_acc
    best_model_wts = copy.deepcopy(model.state_dict())
    torch.save(model.state_dict(), "best_model.pth")
    print(" Best model updated and saved!")
    early_stop_counter = 0 # reset early stopping
else:
    early_stop_counter += 1

print(f"Epoch [{epoch+1}/{epochs}] "
      f"Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f} | "
      f"Train Acc: {train_acc:.2f}% | Val Acc: {val_acc:.2f}%")

# ----- Early Stopping -----
if early_stop_counter >= patience:
    print(f"■ Early stopping at epoch {epoch+1}")
    break

# Load best weights
model.load_state_dict(best_model_wts)
print(f" Training Finished. Best Val Acc: {best_val_acc:.2f}%")
```

```
Epoch 1 ===== Training Started ----
1099
1099 pass done!ge 1099 Loaded Best model updated and saved!
Epoch [1/50] Train Loss: 1.8664 | Val Loss: 0.9817 | Train Acc: 45.02% | Val Acc: 70.75%
Epoch 2 ===== Training Started ----
1099
1099 pass done!ge 1099 Loaded Best model updated and saved!
Epoch [2/50] Train Loss: 1.2059 | Val Loss: 0.7287 | Train Acc: 62.98% | Val Acc: 79.51%
Epoch 3 ===== Training Started ----
1099
1099 pass done!ge 1099 Loaded Best model updated and saved!
Epoch [3/50] Train Loss: 1.0349 | Val Loss: 0.5647 | Train Acc: 68.03% | Val Acc: 83.63%
Epoch 4 ===== Training Started ----
1099
1099 pass done!ge 1099 LoadedEpoch [4/50] Train Loss: 0.8888 | Val Loss: 0.5765 | Train Acc: 72.68% | Val Acc: 82.12%
Epoch 5 ===== Training Started ----
1099
1099 pass done!ge 1099 Loaded Best model updated and saved!
Epoch [5/50] Train Loss: 0.7924 | Val Loss: 0.4110 | Train Acc: 75.33% | Val Acc: 87.37%
Epoch 6 ===== Training Started ----
1099
1099 pass done!ge 1099 LoadedEpoch [6/50] Train Loss: 0.7225 | Val Loss: 0.4481 | Train Acc: 77.40% | Val Acc: 85.39%
Epoch 7 ===== Training Started ----
1099
1099 pass done!ge 1099 Loaded Best model updated and saved!
Epoch [7/50] Train Loss: 0.6824 | Val Loss: 0.3565 | Train Acc: 78.59% | Val Acc: 89.31%
Epoch 8 ===== Training Started ----
1099
1099 pass done!ge 1099 Loaded Best model updated and saved!
Epoch [8/50] Train Loss: 0.6429 | Val Loss: 0.3319 | Train Acc: 79.89% | Val Acc: 90.01%
Epoch 9 ===== Training Started ----
1099
1099 pass done!ge 1099 LoadedEpoch [9/50] Train Loss: 0.6155 | Val Loss: 0.3383 | Train Acc: 80.82% | Val Acc: 89.49%
Epoch 10 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [10/50] Train Loss: 0.5829 | Val Loss: 0.2902 | Train Acc: 81.58% | Val Acc: 91.17%
Epoch 11 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [11/50] Train Loss: 0.5641 | Val Loss: 0.2846 | Train Acc: 82.12% | Val Acc: 90.82%
Epoch 12 ===== Training Started ----
```

```
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [12/50] Train Loss: 0.5429 | Val Loss: 0.2565 | Train Acc: 82.94% | Val Acc: 91.84%
Epoch 13 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [13/50] Train Loss: 0.5239 | Val Loss: 0.2797 | Train Acc: 83.45% | Val Acc: 91.24%
Epoch 14 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [14/50] Train Loss: 0.5102 | Val Loss: 0.2525 | Train Acc: 83.84% | Val Acc: 91.92%
Epoch 15 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [15/50] Train Loss: 0.4927 | Val Loss: 0.2427 | Train Acc: 84.33% | Val Acc: 92.27%
Epoch 16 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [16/50] Train Loss: 0.4863 | Val Loss: 0.2553 | Train Acc: 84.57% | Val Acc: 91.73%
Epoch 17 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [17/50] Train Loss: 0.4659 | Val Loss: 0.2213 | Train Acc: 85.22% | Val Acc: 92.98%
Epoch 18 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [18/50] Train Loss: 0.4588 | Val Loss: 0.2135 | Train Acc: 85.52% | Val Acc: 93.30%
Epoch 19 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [19/50] Train Loss: 0.4529 | Val Loss: 0.2142 | Train Acc: 85.64% | Val Acc: 92.96%
Epoch 20 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [20/50] Train Loss: 0.4409 | Val Loss: 0.2112 | Train Acc: 85.96% | Val Acc: 93.15%
Epoch 21 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [21/50] Train Loss: 0.4260 | Val Loss: 0.1971 | Train Acc: 86.38% | Val Acc: 93.73%
Epoch 22 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [22/50] Train Loss: 0.4250 | Val Loss: 0.2131 | Train Acc: 86.41% | Val Acc: 93.12%
Epoch 23 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [23/50] Train Loss: 0.4110 | Val Loss: 0.1908 | Train Acc: 86.83% | Val Acc: 93.68%
```

```
Epoch 24 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [24/50] Train Loss: 0.4069 | Val Loss: 0.2104 | Train Acc: 87.14% | Val Acc: 93.05%
Epoch 25 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [25/50] Train Loss: 0.3991 | Val Loss: 0.1845 | Train Acc: 87.24% | Val Acc: 93.82%
Epoch 26 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [26/50] Train Loss: 0.3930 | Val Loss: 0.2105 | Train Acc: 87.46% | Val Acc: 93.36%
Epoch 27 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [27/50] Train Loss: 0.3909 | Val Loss: 0.1730 | Train Acc: 87.28% | Val Acc: 94.55%
Epoch 28 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [28/50] Train Loss: 0.3864 | Val Loss: 0.1729 | Train Acc: 87.63% | Val Acc: 94.47%
Epoch 29 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [29/50] Train Loss: 0.3834 | Val Loss: 0.1807 | Train Acc: 87.70% | Val Acc: 94.24%
Epoch 30 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [30/50] Train Loss: 0.3737 | Val Loss: 0.1773 | Train Acc: 88.15% | Val Acc: 94.41%
Epoch 31 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [31/50] Train Loss: 0.3633 | Val Loss: 0.1619 | Train Acc: 88.26% | Val Acc: 94.84%
Epoch 32 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [32/50] Train Loss: 0.3647 | Val Loss: 0.1670 | Train Acc: 88.34% | Val Acc: 94.57%
Epoch 33 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [33/50] Train Loss: 0.3593 | Val Loss: 0.1653 | Train Acc: 88.33% | Val Acc: 94.68%
Epoch 34 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [34/50] Train Loss: 0.3521 | Val Loss: 0.1539 | Train Acc: 88.57% | Val Acc: 95.13%
Epoch 35 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [35/50] Train Loss: 0.3479 | Val Loss: 0.1525 | Train Acc: 88.71% | Val Acc: 94.98%
Epoch 36 ===== Training Started ----
```

```
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [36/50] Train Loss: 0.3441 | Val Loss: 0.1441 | Train Acc: 89.01% | Val Acc: 95.39%
Epoch 37 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [37/50] Train Loss: 0.3492 | Val Loss: 0.1586 | Train Acc: 88.73% | Val Acc: 94.91%
Epoch 38 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [38/50] Train Loss: 0.3415 | Val Loss: 0.1528 | Train Acc: 89.01% | Val Acc: 94.97%
Epoch 39 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [39/50] Train Loss: 0.3391 | Val Loss: 0.1730 | Train Acc: 88.95% | Val Acc: 94.49%
Epoch 40 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [40/50] Train Loss: 0.3329 | Val Loss: 0.1704 | Train Acc: 89.30% | Val Acc: 94.57%
Epoch 41 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [41/50] Train Loss: 0.3379 | Val Loss: 0.1615 | Train Acc: 89.11% | Val Acc: 94.85%
Epoch 42 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [42/50] Train Loss: 0.3255 | Val Loss: 0.1670 | Train Acc: 89.44% | Val Acc: 94.71%
Epoch 43 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [43/50] Train Loss: 0.3305 | Val Loss: 0.1468 | Train Acc: 89.36% | Val Acc: 95.42%
Epoch 44 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [44/50] Train Loss: 0.3304 | Val Loss: 0.1487 | Train Acc: 89.41% | Val Acc: 95.14%
Epoch 45 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [45/50] Train Loss: 0.3141 | Val Loss: 0.1374 | Train Acc: 89.82% | Val Acc: 95.56%
Epoch 46 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [46/50] Train Loss: 0.3221 | Val Loss: 0.1536 | Train Acc: 89.55% | Val Acc: 95.04%
Epoch 47 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [47/50] Train Loss: 0.3193 | Val Loss: 0.1552 | Train Acc: 89.60% | Val Acc: 95.27%
Epoch 48 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [48/50] Train Loss: 0.3200 | Val Loss: 0.1507 | Train Acc: 89.66% | Val Acc: 95.19%
```

```
Epoch 49 ===== Training Started ----
1099
1099 pass done!age 1099 LoadedEpoch [49/50] Train Loss: 0.3149 | Val Loss: 0.1709 | Train Acc: 89.82% | Val Acc: 94.66%
Epoch 50 ===== Training Started ----
1099
1099 pass done!age 1099 Loaded Best model updated and saved!
Epoch [50/50] Train Loss: 0.3135 | Val Loss: 0.1347 | Train Acc: 89.76% | Val Acc: 95.62%
Training Finished. Best Val Acc: 95.62%
```

Saving the model using torchscript

```
In [49]: scripted_model = torch.jit.script(model)
scripted_model.save("plant_disease_simple_cnn_torchscript.pt")
```

```
In [55]: def imshow(img, title):
    img = img.numpy().transpose((1, 2, 0))
    img = np.clip(img, 0, 1) # normalize
    plt.imshow(img)
    plt.title(title)
    plt.axis("off")

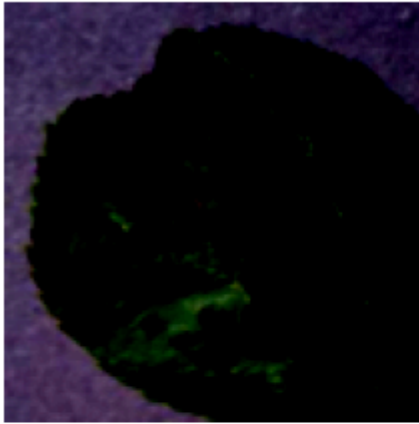
    # Get a batch from validation loader
    dataiter = iter(valid_loader)
    images, labels = next(dataiter)
    images, labels = images.to(device), labels.to(device)

    # Predictions
    model.eval()
    with torch.no_grad():
        outputs = model(images)
        _, preds = torch.max(outputs, 1)

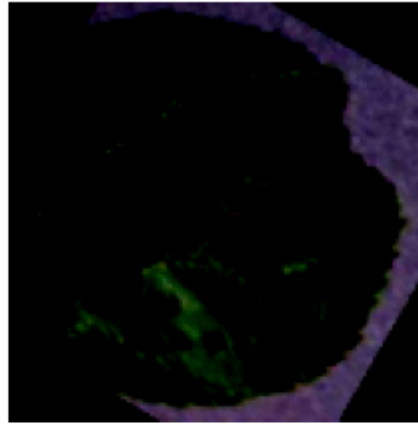
    # Show first 6 images
    plt.figure(figsize=(12, 6))
    for i in range(6):
        ax = plt.subplot(2, 3, i+1)
        title = f"True: {labels[i+7].item()} | Pred: {preds[i].item()}"
        imshow(images[i+7].cpu(), title)
```

```
plt.show()
```

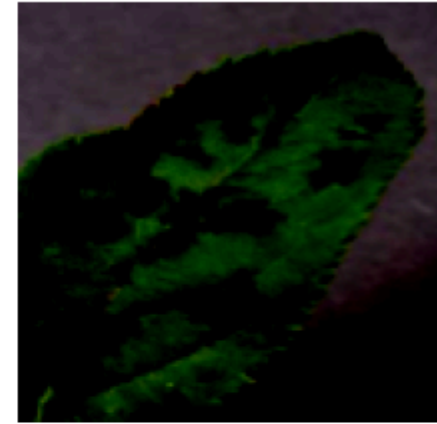
True: 0 | Pred: 0



True: 0 | Pred: 0



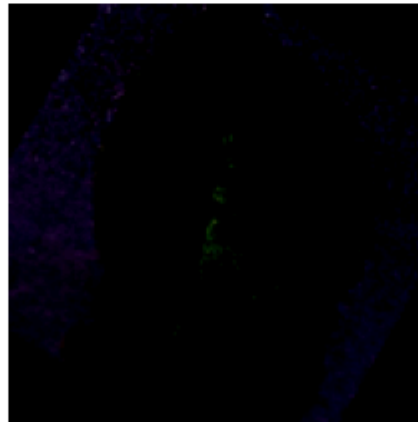
True: 0 | Pred: 0



True: 0 | Pred: 0



True: 0 | Pred: 0



True: 0 | Pred: 0



Confusion Matrix

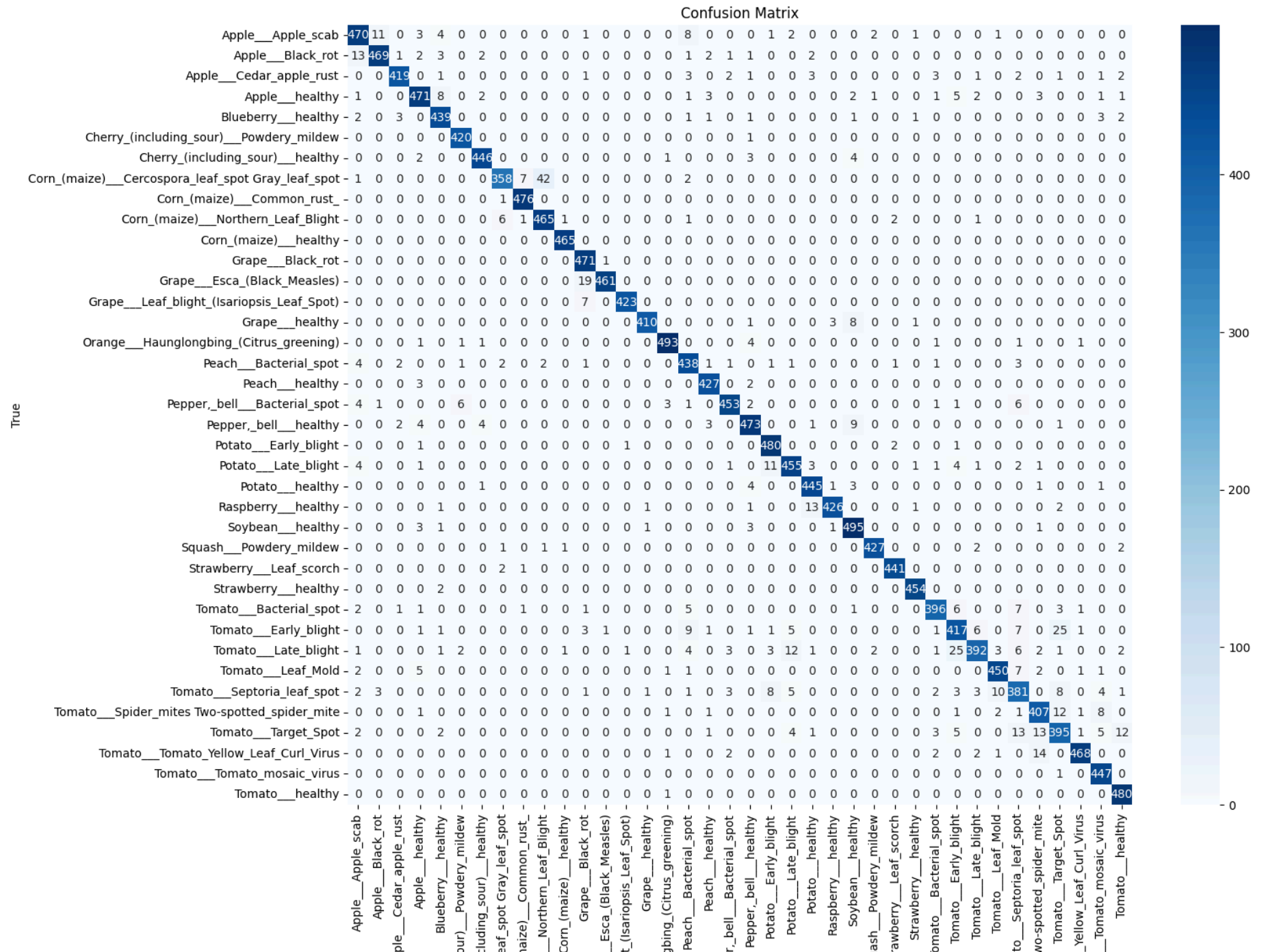
```
In [45]: all_preds = []  
         all_labels = []  
  
         model.eval()
```

```
with torch.no_grad():
    for images, labels in valid_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, preds = torch.max(outputs, 1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

# Confusion Matrix
cm = confusion_matrix(all_labels, all_preds)
classes = valid_loader.dataset.classes

plt.figure(figsize=(15, 12))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=classes, yticklabels=classes)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()
```



Ap
Cherry_(including_s
Cherry_(in
Corn_(maize)____Cercospora_l
Corn_(n
Corn_(maize)
Grape_
Grape__Leaf_blight
Orange__Haunglon
Peppe
Squ
St
T
Toma
Tomato__Spider_mites T
Tomato__Tomato
Tomato_

Predicted

Load Saved Model

```
In [50]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = torch.jit.load("plant_disease_simple_cnn_torchscript.pt", map_location=device)
model.eval()

print("TorchScript model loaded successfully and set to evaluation mode.")
```

TorchScript model loaded successfully and set to evaluation mode.

Model Building: Transfer Learning using ResNet50

```
In [8]: model = models.resnet50(pretrained=True)
num_classes = len(classes)
in_features = model.fc.in_features
model.fc = nn.Linear(in_features, num_classes)
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam([p for p in model.parameters() if p.requires_grad], lr=LR)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=2)
```

```
e:\Pytorch_learning\.venv\Lib\site-packages\torchvision\models\_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.  
    warnings.warn(  
e:\Pytorch_learning\.venv\Lib\site-packages\torchvision\models\_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ResNet50_Weights.IMAGENET1K_V1`. You can also use `weights=ResNet50_Weights.DEFAULT` to get the most up-to-date weights.  
    warnings.warn(msg)
```